

# BASES DE DATOS OBJETO-RELACIONALES Y ORIENTADAS A OBJETOS

# Bases de Datos Objeto-Relacionales y Orientadas a Objetos

El objetivo de esta Unidad es distinguir las ventajas e inconvenientes de las BD que almacenan objetos, identificar los tipos de objetos, ver las características del estándar ODMG y la gestión de una BDOO.

## Bases de Datos Objeto-Relacionales (BDOR).

Las BDOR son una extensión de las Bases de Datos relacionales tradicionales, a las que se le añaden conceptos del modelo orientado a objeto, por lo que un Sistema de Gestión de Base de Datos Objeto-Relacional (SGBDOR) contiene características del modelo relacional y del orientado a objetos, es decir, es un sistema relacional que permite almacenar objetos en las tablas.

# Bases de Datos Objeto-Relacionales

## Características:

Soporte de tipo de datos básicos y complejos. El usuario puede crear sus propios tipos de datos.

Soporte para crear métodos para esos tipos de datos.

Herencia

Se pueden almacenar múltiples valores en una columna de una misma fila

Relaciones (tablas) anidadas

Compatibilidad con las BDR tradicionales (las aplicaciones no tienen por qué ser reescritas para el nuevo modelo)

El **inconveniente** es que aumenta la complejidad del sistema (y aumento de coste)

# Bases de Datos Objeto-Relacionales

*(Ejemplos para ORACLE ;-)* Oracle es una de las BD que ha evolucionado incorporando extensiones que le provee de mecanismos para la inclusión de objetos y la asociación de métodos a estos objetos.

## **Tipos de Objetos.**

Para crear tipos de objetos => CREATE TYPE

Para borrar tipos => DROP TYPE

## **Métodos.**

Para Oracle existe un constructor predefinido por cada objeto, pero se pueden sobrescribir o crear constructores adicionales.

Los **constructores** llevarán en la cláusula RETURN la expresión **RETURN SELF AS RESULT**

# Bases de Datos Objeto-Relacionales

## ORACLE® Application Express

[Home](#) [Application Builder](#) [SQL Workshop](#) [Team](#)

Home > SQL Workshop > SQL Commands

☒ Autocommit Rows   

```
create or replace type direccion as object (  
calle varchar2(25),  
ciudad varchar2 (20),  
codigo number(5));
```

[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

Type created.

0.59 seconds

☒ Autocommit Rows   

```
create or replace type persona as object (  
codigo number,  
nombre varchar2 (30),  
domicilio direccion,  
fecha_nac date);
```

## Ejemplo de métodos:

## ORACLE® Application Express

[Home](#) [Application Builder](#) [SQL Workshop](#) [Team D](#)

Home > SQL Workshop > SQL Commands

☒ Autocommit Rows   

```
CREATE OR REPLACE TYPE "DIRECCION" as object (  
calle varchar2(25),  
ciudad varchar2 (20),  
codigo number(5),  
MEMBER PROCEDURE set_ciudad(c varchar2),  
MEMBER FUNCTION get_ciudad return varchar2);
```

[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

Type created.

0.34 seconds

# Bases de Datos Objeto-Relacionales

## Ejemplo de Constructor:

ORACLE® Application Express

[Home](#) [Application Builder ▼](#) [SQL Workshop ▼](#) [Team Development ▼](#) [Administration ▼](#)

[Home](#) > [SQL Workshop](#) > [SQL Commands](#)

☒ Autocommit Rows    [Save](#) [Run](#)

```
CREATE OR REPLACE TYPE "DIRECCION" as object (  
  calle varchar2(25),  
  ciudad varchar2 (20),  
  codigo number(5),  
  CONSTRUCTOR FUNCTION direccion (calle varchar2, ciudad varchar2, codigo number)RETURN SELF AS RESULT);
```

[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

Type created.

0.14 seconds

Una vez declarados los objetos se podrán inicializar y utilizar como cualquier otro tipo predefinido y usarlos en bloques PL/SQL

# Bases de Datos Orientadas a Objetos

## **Bases de Datos Orientadas a Objetos (BDOO).**

Las BDOO son aquellas cuyo modelo de datos está orientado a objetos, soportando el almacenamiento de métodos y datos. Un Sistema Gestor de Base de Datos Orientada a Objetos (SGBDOO) es un sistema gestor de base de datos que almacena objetos.

# Bases de Datos Orientadas a Objetos

## Características:

Los datos se almacenan como objetos

Cada objeto se identifica mediante un identificador único **OID** (*Object Identifier*) y no puede ser modificado por el usuario.

Cada objeto define sus métodos y atributos y la interfaz mediante la cual se puede acceder a él.

Debe cumplir las características de un SGBD:

- Persistencia
- Concurrencia
- Recuperaciones ante fallos
- Gestión de almacenamiento secundario
- Facilidad de consultas
- Encapsulación, identidad, herencia y polimorfismo

# Bases de Datos Orientadas a Objetos

## Ventajas:

- Mayor capacidad de modelado (representación de datos de forma natural)
- Extensibilidad (construir nuevos tipos a partir de existentes)
- Existe una única interfaz entre el DML y el lenguaje de programación (no hay que incrustar SQL en lenguaje Java)
- Lenguaje de consultas más expresivo
- Soporte de transacciones largas
- Adecuación de aplicaciones avanzadas de BD (CASE, CAD, Multimedia...)

## Inconvenientes:

- Falta de un modelo de datos universal (carecen de base teórica) y falta de estándares
- Falta de experiencia (uso aún limitado) y competencia con los SGBDR y SGBDOR
- La optimización de consultas compromete la encapsulación
- Complejidad
- Falta de soporte a las vistas
- Falta de soporte a la seguridad

# Bases de Datos Orientadas a Objetos

## Qué hay que tener en cuenta :

- Los SGBDR siguen siendo más potentes en cuanto a la seguridad. Las BDOO utilizan los propios recursos del SO, o bien, utilizan otros mecanismos de protección con password, etc... pero con escaso nivel de protección a nivel de objeto, miembro... Deben “desarrollarla”
- Los SGBDOO no soportan restricciones ni vistas, deben implementarlas con operaciones.
- Y lo mismo ocurre con **la integridad referencial**

# Bases de Datos Orientadas a Objetos

## El estándar ODMG (*Object Database Management Group*)

### Componentes:

- 1) Modelo de objetos
- 2) Lenguaje de Definición de Objetos (ODL)
- 3) Lenguaje de Consulta de Objetos (OQL)
- 4) Conexión con los lenguajes C++, Java...

### Modelo de objetos

### Tipos de Objetos:

- a) Atómicos: boolean, short, long, char, string, float, double...
- b) Tipos estructurados: date, time...
- c) Colecciones `<interfaceCollection>`:
  - `set<tipo>`: grupo desordenado de objetos del mismo tipo que no permite duplicados
  - `bag<tipo>`: grupo desordenado de objetos del mismo tipo que permite duplicados
  - `list<tipo>`: grupo ordenado de objetos del mismo tipo que permite duplicados
  - `array<tipo>`: grupo ordenado de objetos del mismo tipo a los que se puede acceder por su posición y su tamaño es dinámico.

# Bases de Datos Orientadas a Objetos

## Clases:

Mediante las clases se especifica el estado y comportamiento de un tipo de objeto.

Pueden incluir métodos.

## Lenguaje ODL:

Equivalente al DDL de los SGBDR.

## Lenguaje OQL:

El lenguaje estándar de consultas de las BDOO, declarativo similar al SQL, con acceso a las propiedades y métodos de los objetos. Sólo incluye operaciones de consulta, las de modificación se realizan mediante los propios métodos.

La sintaxis básica es:

*SELECT <lista de valores>*

*FROM <lista de colecciones y miembros>*

*WHERE <condición>*

# Bases de Datos Orientadas a Objetos

Comparación entre los diferentes SGBD =>

## BDR

Utiliza clave primaria

No puede crear nuevos tipos de datos

Utiliza tablas

Utiliza tecnología de BDR

En consultas utiliza SQL  
(SELECT FROM WHERE)

No puede manipular datos complejos

## BDOO

Utiliza OID

Puede crear nuevos tipos de datos

Utiliza “objetos”

Utiliza técnicas de POO

En consultas utiliza SQL, pero se llama OQL (SELECT FROM WHERE)

Puede manipular datos complejos

## BDOR

Utiliza alias para cada tabla

Puede crear nuevos tipos de datos

Utiliza tablas

Combina POO con BDR

Para consultas utiliza un API separado que mapea las tablas (SQL3)

Puede manipular datos complejos

# Bases de Datos Orientadas a Objetos

## La Integridad Referencial en Db4o:

Una forma de crear un modelo de integridad referencial es mediante la gestión de los eventos de interacción con la BD. Db4o dispone de la interfaz EventRegistry, que permite crear un gestor para los eventos relacionados con la persistencia, facilitando la labor de cancelar una inserción o desencadenar un borrado en cascada ante el borrado de un objeto.

```
EventRegistry eventos;
```

```
...
```

```
ObjectContainer db = Db4oEmbedded.openFile(Db4oEmbedded.newConfiguration(), "bdpersonas.yap");
```

```
eventos=EventRegistryFactory.forObjectContainer(db);
```

Se añadirán los correspondientes addListener, a cada proceso, de borrado e inserción para controlarlos, como si se tratase de triggers... pero como hemos mencionado, todo esto hay que programarlo.

# Bases de Datos Orientadas a Objetos

## Relaciones en BDOO

Las bases de datos relacionales representan las relaciones mediante las claves ajenas. Las relaciones se utilizan para hacer concatenaciones (*join*) de tablas. Por el contrario, las bases de datos orientadas a objetos implementan sus relaciones incluyendo en cada objeto los identificadores de los objetos con los que se relaciona (OID).

Un **identificador de objeto** es un atributo interno que posee cada objeto. Ni los programadores, ni los usuarios que realizan consultas de forma interactiva, ven o manipulan estos identificadores directamente. Los identificadores de los objetos los asigna el SGBD y es el único que los utiliza.

# Bases de Datos Orientadas a Objetos

El modelo orientado a objetos permite los atributos **multivaluados**, agregaciones a las que se denomina conjuntos (sets).

Para crear una relación de ***uno a muchos***, se define un atributo en la parte del uno que será de la clase del objeto con el que se relaciona. Este atributo contendrá el identificador de objeto del padre. La clase del objeto padre contendrá un atributo que almacenar a un conjunto de valores: los identificadores de los objetos hijo con los que se relaciona. Cuando el SGBD ve que un atributo tiene como tipo de datos una clase, ya sabe que el atributo contendrá un identificador de objeto => del tipo ***relationship*** (en lenguaje ODL).

# Bases de Datos Orientadas a Objetos

Las relaciones de **muchos a muchos** se pueden representar directamente en las bases de datos orientadas a objetos, sin necesidad de crear entidades intermedias. Para representar la relación, *cada clase que participa en ella define un atributo que contendrá un conjunto de valores de la otra clase con la que se relaciona*. Aunque el hecho de poder representar relaciones de muchos a muchos parece aportar muchas ventajas, hay que tener mucho cuidado cuando se utilizan.

- 1) En primer lugar, si la relación tiene otros datos, será necesario crear una entidad intermedia que contenga estos datos. Por ejemplo, en la relación de los artículos con los proveedores, en donde cada proveedor puede tener un precio distinto para un mismo artículo. En este caso, la relación de muchos a muchos se sustituye por dos relaciones de uno a muchos, como se haría en una base de datos relacional.
- 2) En segundo lugar, se puede diseñar una base de datos que contiene relaciones de muchos a muchos en donde o bien se pierde información, o bien se hace imposible determinar las relaciones con precisión. También en estos casos la solución es incluir una entidad intermedia que represente la relación.

# Bases de Datos Orientadas a Objetos



## ***Integridad de las Relaciones***

Para que las relaciones funcionen en una base de datos orientada a objetos pura, los identificadores de los objetos deben corresponderse en ambos extremos de la relación.